

ASE WEEK 02 — UNIT 04

Incremental Development

Delivering in slices, letting each slice absorb change — and the two things it costs you

Sources. Sommerville, *Software Engineering* 9th ed., pp.47 and 49–51, with pp.64–65 for incremental delivery. Mall, *Fundamentals of Software Engineering* 4th ed., pp.95–97. **Page numbers are PDF page numbers**, not printed page numbers. **Method.** Every definition is quoted verbatim from the source before it is explained, and every claim carries a page anchor. Where a concept rests on a single source, or where a phase is summarised rather than reproduced in full, the coverage note says so explicitly.

Where this sits

Previous unit: Unit 03 ended on a **limitation of prototyping** that sets up this file precisely. Mall's balance sheet: prototyping is "*effective only for those projects for which the risks can be identified upfront before the development starts*", and because the prototype is "*constructed only at the start of the project*", it is "*ineffective for risks identified later during the development cycle*" (Mall p.95).

So the gap is this: prototyping buys knowledge **once**, at the beginning. What if instead we delivered the **real** system in slices — so that **every slice** buys knowledge, not just the first?

The problem this file opens with: *what if we delivered working software early, in slices, and let each slice absorb the change?*

The question it hands to Unit 05: incremental development absorbs change well — but it is **slow** (you wait for the last version), the **process is not visible**, and the **structure degrades**. If the **deadline** is the real constraint rather than the requirements, you need a different answer. That answer is RAD.

1. What incremental development is

1.1 Sommerville's definition

Verbatim (Sommerville p.49): "*Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and evolving it through several versions until an adequate system has been developed.*"

Verbatim (Sommerville p.50): "*Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.*"

الفكرة الأساسية عند Sommerville: بناء تنفيذ أولي، وعرضه على المستخدم للتعليق، وتطويره عبر عدة نسخ لحد ما يُطوّر نظام ملائم.

والنقطة الآلية المهمة: التوصيف والتطوير والتحقق متداخلة، مو منفصلة — مع تغذية راجعة سريعة بين الأنشطة.

قارن مع الملف 01: هناك شفتنا إن الأنشطة الأربعة موجودة بكل عملية. وهنا نشوف الفرق بين النماذج: Waterfall يفرزهن مراحل منفصلة، والتزايدي يداخلهن.
احفظ كلمة **interleaved** — هي الكلمة اللي تميّز النموذج.

1.2 Mall's definition — and the second name

Verbatim (Mall p.95): "This life cycle model is sometimes referred to as the successive versions model and sometimes as the incremental model. In this life cycle model, first a simple working system implementing only a few basic features is built and delivered to the customer. Over many successive iterations successive versions are implemented and delivered to the customer until the desired system is realised."

الاسم الثاني: النموذج يُسمّى أحياناً **successive versions model** وأحياناً **incremental model** — تسميتان لنفس النموذج. (وهذا مثل exploratory / build and fix / code and fix بالمفهوم السابق — نمط متكرر عند Mall).
والتعريف: نظام عامل بسيط يُبنى ويُسلّم، وبعدين نسخ متتالية تُنفَّذ وتُسلّم لحد تحقيق النظام المطلوب.
لاحظ الفرق عن النموذج الأولي : - النموذج الأولي: نظام خام ولعبة (toy and crude)، يُرمى. - التزايدي: نظام عامل (working) بسيط، يُسلّم ويُستخدم — ما يُرمى.
وهذا هو الفرق الجوهرى: التزايدي يسلم الحقيقي بأجزاء؛ الأولي يسلم الوهمي كامل.

1.3 Why it matches how humans actually work

Verbatim (Sommerville p.50): "Incremental development reflects the way that we solve problems. We rarely work out a complete problem solution in advance but move toward a solution in a series of steps, backtracking when we realize that we have made a mistake. By developing the software incrementally, it is cheaper and easier to make changes in the software as it is being developed."

هذي أهم حجّة مفاهيمية بالنموذج:

التطوير التزايدي يعكس الطريقة اللي نحلّ بيها المشاكل. نادراً ما نطلع بحل كامل مقدماً — بل نتقدّم بحلّ على خطوات، ونرجع لورا لما نكتشف إننا غلطنا.

ولهذا التطوير التزايدي يجعل التغيير أرخص وأسهل.

لبش هذا مهم للامتحان؟ لأنه مو حجّة تقنية — حجّة إنسانية. والنماذج اللي بعدها (خصوصاً Agile) تبني على نفس المنطق. ولو سألك «لبش التزايدي طبيعي؟» — الجواب: لأنه يعكس طريقة حلنا للمشاكل.

1.4 And it is now the default

Verbatim (Sommerville p.50): "Incremental software development, which is a fundamental part of agile approaches, is better than a waterfall approach for most business, e-commerce, and personal systems."

Verbatim (Sommerville p.51): "Incremental development in some form is now the most common approach for the development of application systems. This approach can be either plan-driven, agile, or, more usually, a mixture of these approaches."

التزايدى جزء أساسى من المقاربات الرشيقة (agile)، وهو أفضل من الـ Waterfall لأغلب الأنظمة التجارية والتجارة الإلكترونية والشخصية.

والأهم: التطوير التزايدى بشكله العام هو الآن أكثر المقاربات شيوعاً لتطوير أنظمة التطبيقات. ويمكن أن يكون: - plan-driven (المدرج مقدماً)، - أو agile، - أو خليط منهما — وهذا الأكثر شيوعاً.

وهذا يعطينا معلومة مهمة: التزايدى هو نموذج واحد — هو إطار يقدر يشتغل بالنمطين. وهذا يفسر ليش Agile مبني عليه.

2. The mechanics — how the slicing works

2.1 Which functionality goes into which increment

Verbatim (Sommerville p.50): "Each increment or version of the system incorporates some of the functionality that is needed by the customer. Generally, the early increments of the system include the most important or most urgently required functionality."

Verbatim (Sommerville p.50): "This means that the customer can evaluate the system at a relatively early stage in the development to see if it delivers what is required. If not, then only the current increment has to be changed and, possibly, new functionality defined for later increments."

كل زيادة تحتوي بعض الوظائف المطلوبة. والزيادات المبكرة تحتوي أهم الوظائف أو الأكثر إلحاحاً.

والنتيجة العملية — وهي ميزة كبيرة: العميل يقدر يقيم النظام بمرحلة مبكرة ليشوف هل يسلم المطلوب. وإذا لا — تُغيّر الزيادة الحالية فقط، وربما تُعرّف وظائف جديدة للزيادات اللاحقة.

قارن بالـ Waterfall: لو المطلوبات غلط، يُعاد كل شي (لأن المرحلة مقفلة والمراحل اللاحقة بنتها). هون تُغيّر شريحة وحدة.

وهذا بالضبط معنى change tolerance اللي شفهنا بالمفهوم السابق 15: التغيير يقع بالجزء الصغير اللي لسه ما انبنى أو اللي الحالى.

2.2 Mall's core / non-core distinction — the ordering rule

Verbatim (Mall p.96): "The development team first undertakes to develop the core features of the system. The core or basic features are those that do not need to invoke any services from the other features. On the other hand, non-core features need services from the core features."

Verbatim (Mall p.96): "Once the initial core features are developed, these are refined into increasing levels of capability by adding new functionalities in successive versions."

Verbatim (Mall p.96): "After the requirements gathering and specification, the requirements are split into several versions. Starting with the core (version 1), in each successive increment, the next version is constructed using an iterative waterfall model of development and deployed at the customer site. After the last (shown as version n) has been developed and deployed at the client site, the full software is deployed."

قاعدة الترتيب عند Mall: يبلشون بالميزات الأساسية (core features) — وهي اللي ما تحتاج تستدعي خدمات من ميزات ثانية. أما غير الأساسية (non-core) فتحتاج خدمات من الأساسية.

وهذا تعريف دقيق ومهم: الأساسية = مستقلة. يعني يُبنى الأساس أولاً لأنه لا يعتمد على غيره.

وبعدها: الميزات الأساسية تُصَفَّى لمستويات قدرة متزايدة بإضافة وظائف جديدة بالنسخ المتتالية.

والتسلسل: بعد جمع المطلوبات وتوصيفها، تُقسَّم المطلوبات على نسخ، ويُبلش بالأساسية (نسخة 1)، وكل نسخة تُبنى بـ **iterative waterfall** وتُنشر بموقع العميل، وبعد النسخة الأخيرة يُنشر البرنامج الكامل.

احفظ القاعدتين: 1. الأساسية أولاً — وهي اللي لا تعتمد على غيرها. 2. كل نسخة تُبنى بـ **waterfall iterative** — يعني التزايد يستخدم نموذجاً ثانياً بالداخل.

وهذي نقطة مهمة للفهم: النماذج متنافية — النموذج التزايدوي يحتوي الـ waterfall iterative بداخله. (وSommerville يقولها بالمفهوم السابق §6: «النماذج ما هي متبادلة الاستبعاد وغالباً تُستخدم معاً».)

2.3 The feedback loop

Verbatim (Mall p.96): "As each successive version of the software is constructed and delivered to the customer, the customer feedback is obtained on the delivered version and these feedbacks are incorporated in the next version. Each delivered version of the software incorporates additional features over the previous version and also refines the features that were already delivered."

مع كل نسخة تُسلَّم، تُجمع التغذية الراجعة وتُدمج بالنسخة التالية. وكل نسخة تضيف ميزات جديدة + تصفّي الميزات المسلّمة سابقاً.

لاحظ الاتجاهين: - إضافة (additional features) — شي جديد - تصفية (refines) — تحسين اللي موجود

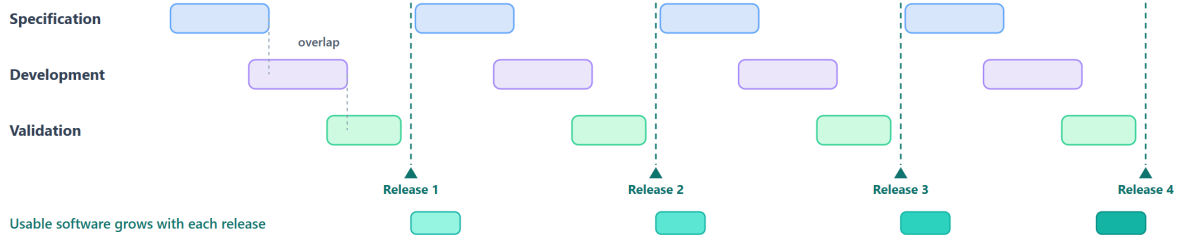
يعني التغذية الراجعة مو بس «شنو ناقص» — كذلك «شنو يحتاج تحسين».

2.6 The whole model in one picture

Incremental Development: Delivering in Slices

Specification, development and validation interleaved — and what that costs the manager (Sommerville pp. 49–51)

THE THREE ACTIVITIES ARE INTERLEAVED, NOT SEQUENTIAL



THE THREE BENEFITS

1 · Customer value early

Usable software is delivered at each release, so the customer gets value long before the last increment.

2 · Lower risk of total failure

A change is absorbed between two increments, not as expensive rework after a completed build.

3 · Priority gets tested most

The highest-priority services are built and exercised first, so they receive the most testing effort.

THE TWO MANAGEMENT PROBLEMS

1 · The process is not visible

Managers need regular deliverables to judge progress. Here the deliverable exists only every few weeks — so documenting every version costs heavily.

2 · Structure degrades as change accumulates

Regular change corrupts the code structure; the cure is continuous **refactoring**, which itself costs money and time. Left alone, the system becomes progressively harder to change.

⚠ EXAM TRAP — incremental *development* is not incremental *delivery*

Development: the increments are the internal slices of a single build (the diagram above).

Delivery: the increments are actually deployed to the customer, who may still receive the complete system only at the end.

Source: I. Sommerville, Software Engineering (9th ed.), pp. 49–51; R. Mall, Fundamentals of Software Engineering (4th ed.), §2.2.5.

Master Studio · Advanced Software Engineering (CS504) · University of Wasit

Incremental development: specification, development and validation interleaved across four increments, with the benefits, the management problems, and the development-versus-delivery distinction

كيف تقرأ الرسم — وهذا خريطة الوحدة كلها:

معناه الهندسي	العنصر في الرسم
validation · development · specification: الأنشطة الثلاثة	المسارات الثلاثة (أزرق · بنفسجي · أخضر)
هذا هو جوهر النموذج: ماكو تسلسل — التخطيط للزيادة الثانية يبيلش قبل ما تخلص اختبار الأولى	التراكب بين المسارات
نقاط التسليم — كل واحد منها نظام شغّال فعلاً	المثلثات + Release 1..4
البرنامج الشغّال يكبر مع كل إصدار — مو مجرد وثائق	الشريط السفلي المتدرّج
قيمة مبكرة · خطر فشل أقل · الأولوية تُختبر أكثر	الصناديق الخضراء (الفوائد الثلاث)
العملية غير مرئية للإدارة · البنية تتدهور مع التغيير	الصناديق الحمراء (المشكلاتان)
التطوير الترايدي ≠ التسليم الترايدي — فرق لازم ما تخلطه	الصندوق الأصفر (فخ الامتحان)

الخلاصة البصرية: النموذج مو خط مستقيم — هو ثلاثة خطوط متوازية متزاحمة، وكل تسليم نقطة على محور الزمن. هذا هو الفرق عن Waterfall كله.

3. The benefits

3.1 Sommerville's three benefits

These are the canonical three, and they are worth memorising as a set.

Verbatim (Sommerville p.50): "Incremental development has three important benefits, compared to the waterfall model: 1. **The cost of accommodating changing customer requirements is reduced.** The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model. 2. *It is easier to get customer feedback on the development work that has been done. Customers can comment on demonstrations of the software and see how much has been implemented. Customers find it difficult to judge progress from software design documents.* 3. **More rapid delivery and deployment of useful software to the customer is possible****, even if all of the functionality has not been included. Customers are able to use and gain value from the software earlier than is possible with a waterfall process."

ثلاث فوائد مقارنة بالWaterfall:

#	الفائدة	التبرير
1	كلفة استيعاب تغيير المتطلبات تقل	لأن كمية التحليل والتوثيق التي تُعاد أقل بكثير
2	تسهيل الحصول على تغذية راجعة	العميل يعلق على عروض توضيحية ويشوف المنجز. والنقطة الذكية: العميل يصعب عليه يقيّم التقدم من مستندات التصميم
3	تسليم ونشر أسرع	حتى لو ما كملت كل الوظائف — العميل يستخدم ويستفيد أبكر

السبب الثاني هو الأعماق: «العميل يصعب عليه يقيّم التقدم من مستندات التصميم» — يعني المستندات مو وسيلة تواصل فعّالة مع العميل. وهذا يفسّر لبش النماذج اللاحقة (Agile) تعتمد على برمجيات عاملة بدل مستندات.

3.2 Mall's two advantages

Verbatim (Mall p.97): "The incremental development model offers several advantages. Two important ones are the following: - **Error reduction:** The **core modules are used by the customer from the beginning** and therefore **these get tested thoroughly**. This reduces chances of errors in the **core modules of the final product**, leading to **greater reliability** of the software. - **Incremental resource deployment:** This model obviates the need for the customer to commit large resources at one go for development of the system. It also saves the developing organisation from deploying large resources and manpower for a project in one go*."

مميزتان عند Mall — وهما مختلفتان عن مميزات Sommerville:

1. **تقليل الأخطاء** — لأن الوحدات الأساسية يستخدمها العميل من البداية، فتُختبر بشكل شامل. وهذا يقلّل الأخطاء بالوحدات الأساسية بالمنتج النهائي → موثوقية أعلى.
2. **نشر الموارد تدريجياً** — يلغي حاجة العميل للالتزام بموارد كبيرة دفعة واحدة، ويوفّر على المؤسسة المطوّرة نشر موارد وقوى عاملة كبيرة دفعة واحدة.

ليش ميزة «نشر الموارد» مهمة؟ لأنها مالية/تجارية مو تقنية. ولو سألك عن فائدة اقتصادية للتزايدى — هذي هي. وهي ما توجد عند Sommerville، والعكس صحيح.

والفرق الأهم: ميزة تقليل الأخطاء عند Mall مختلفة عن «تقليل الكلفة» عند Sommerville — لأنها عن الاختبار الفعلي المبكر، مو عن التوثيق.

احفظ الجدولين معاً — لأن السؤال ممكن يجي «اذكر فوائد التزايدى» والمصدر يحدد الجواب.

4. The two management problems

This is the cost side, and it is where this file turns. Sommerville is precise.

Verbatim (Sommerville p.51): "From a management perspective, the incremental approach has two problems: 1. **The process is not visible.** Managers need **regular deliverables to measure progress.** If systems are developed quickly, it is **not cost-effective to produce documents that reflect every version** of the system. 2. *System structure tends to degrade as new increments are added. Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly*.*"

مشكلتان من منظور الإدارة:

1. العملية غير مرئية (the process is not visible). المديرين يحتاجون مخرجات منتظمة لقياس التقدم. ولو الأنظمة تُطوّر بسرعة، ما يكون مجدياً اقتصادياً إنتاج مستندات تعكس كل نسخة.

2. بنية النظام تميل للتدهور. إلا لو أنفق وقت ومال على إعادة الهيكلة (refactoring)، فإن التغيير المنتظم يفسد بنيته، ويصير دمج تغييرات إضافية أصعب وأكلف تدريجياً.

اربط هذا بالمفهوم السابق: هناك شفتنا متلازمة الـ 99% مكتمل — والسبب كان عدم القدرة على قياس التقدم. وهون نشوف نفس المشكلة ترجع، بس بسبب معاكس: مو لأن المراحل غير محددة، بل لأن التزايدى ما ينتج مستندات كافية للقياس.

يعني: الـ Waterfall مفرط بالتوثيق (عيب 6 بالمفهوم السابق)، والتزايدى ناقص بالتوثيق → ولا واحد منهما يحلّ مشكلة الرؤية الإدارية. وهذا توتر حقيقي بالمجال، ويظهر بحلول وسطية بالنماذج الجاية.

والمشكلة الثانية أعمق تقنياً: تدهور البنية (structure degradation). لأن كل زيادة تضاف على موجود، وإلا صار refactoring، البنية تفسد. وهي نفس المشكلة اللي وصفها Mall بالنموذج التطوري بـ «التصميم العشوائي (ad hoc design)» — يعني نفس العيب يظهر بمصدرين مستقلين.

4.1 Where it gets acute — large systems

Verbatim (Sommerville p.51): "The problems of incremental development become particularly acute for large, complex, long-lifetime systems, where different teams develop different parts of the system. Large systems need a stable framework or architecture and the responsibilities of the different teams working on parts of the system need to be clearly defined with respect to that architecture. This has to be planned in advance rather than developed incrementally."

المشاكل تصير حادة بشكل خاص بالأنظمة الكبيرة، المعقدة، وطويلة العمر، حيث فرق مختلفة تطوّر أجزاء مختلفة.

والسبب: الأنظمة الكبيرة تحتاج إطاراً أو معمارية مستقرة، ومسؤوليات الفرق لازم تكون محدّدة بوضوح بالنسبة لهذا المعمارية. وهذا لازم يُخطّط مسبقاً، مو يُطوّر تزايدياً.

وهذي جملة مفصلية: «هذا لازم يُخطّط مسبقاً» — يعني المعمارية مو شي يتزايد. وهذا يحدّ من صلاحية التزايد.

اربطها بالمفهوم السابق: Sommerville هناك قال: «الأجزاء المفهومة تُوصّف وتُطوّر بالwaterfall؛ والأجزاء الصعبة التحديد مثل واجهة المستخدم تُطوّر دائماً تزايدياً» (p.47). وهون يقول نفس المنطق على مستوى المعمارية: الهيكل الكبير → تخطيط مسبق؛ الأجزاء المتغيّرة → تزايد.

فالقاعدة المجمّعة من الاثنين:

التخطيط المسبق للمعمارية، والتطوير التزايدى للمحتوى.

5. Incremental *development* versus incremental *delivery* — the distinction that carries marks

This is the sharpest distinction in the file, and it is easy to miss because the two share a name.

Verbatim (Sommerville p.51): "You can develop a system incrementally and expose it to customers for comment, without actually delivering it and deploying it in the customer's environment. Incremental delivery and deployment means that the software is used in real, operational processes. This is not always possible as experimenting with new software can disrupt normal business processes."

الفرق الحاسم:

Incremental delivery	Incremental development	
نعم	نعم	النظام يُبنى بزيادات؟
نعم — ويستخدمه	نعم — للتعليق	العميل يشوف؟
نعم — يُستخدم بعمليات حقيقية	لا	يُنشر بيئة العميل؟
مو دائماً ممكن — لأن تجربة برنامج جديد قد تعطل عمليات العمل العادية	ممكن دائماً	الشرط

الجملة اللي تنحفظ: «يمكن تطوير نظام تزايدياً وعرضه على العملاء للتعليق، بلا تسليم فعلي ونشر بيئة العميل. أما التسليم التزايدى فيعني إن البرنامج يُستخدم بعمليات حقيقية تشغيلية.»

وعلته: تجربة البرنامج الجديد قد تعطل عمليات العمل العادية — ولهذا مو دائماً ممكن.

ليش هذا فرق مهم؟ لأنه يحدّد صلاحية النموذج: التطوير التزايدى ممكن دائماً تقريباً، أما التسليم التزايدى فمشروط بقدرة المؤسسة على تحمل التعطيل.

5.1 Incremental delivery in full

Verbatim (Sommerville p.64): "Incremental delivery... is an approach to software development where some of the developed increments are delivered to the customer and deployed for use in an operational environment. In an incremental delivery process, customers identify, in outline, the services to be provided by the system. They identify which of the services are most important and which are least important to them. A number of delivery increments are then defined, with each increment providing a sub-set of the system functionality. The allocation of services to increments depends on the service priority, with the highest-priority services implemented and delivered first."

Verbatim (Sommerville p.64): "Once the system increments have been identified, the requirements for the services to be delivered in the first increment are defined in detail and that increment is developed. During development, further requirements analysis for later increments can take place but requirements changes for the current increment are not accepted."

كيف يشتغل التسليم التزايدى: 1. العملاء يحددون بشكل عام الخدمات التي يقدمها النظام، ويميّزون الأهم من الأقل أهمية. 2. تُعرّف عدة زيادات تسليم، كل واحدة تقدّم مجموعة فرعية من وظائف النظام. 3. توزيع الخدمات على الزيادات يعتمد على الأولوية — الأولوية الأعلى تُنفَّذ وتُسلّم أولاً.

والقاعدة الحرجة: «أثناء التطوير، ممكن يجري تحليل مطلوبات إضافي للزيادات اللاحقة، بس تغييرات المطلوبات للزيادة الحالية ما تُقبل.»

هذي جملة مهمة: التسليم التزايدى مو فوضى — فيه تجميد موضعي: الزيادة الحالية مقفلة، واللاحقة مفتوحة. وهذا فرق دقيق عن التطويري اللي ما يجمد شي.

فصار عندنا تسلسل دقيق:

النموذج	شنو يتجمّد؟
Waterfall	كل شي، من البداية
Incremental	المطلوبات الكاملة أولاً، ثم الزيادة الحالية
Evolutionary	لا شي

احفظ هذا التدرّج — هو يعطيك إجابة دقيقة لأي سؤال «شنو الفرق؟».

5.2 The three advantages of incremental delivery

Verbatim (Sommerville p.64): "Incremental delivery has a number of advantages: 1. **Customers can use the early increments as prototypes and gain experience that informs their requirements for later system increments. Unlike prototypes, these are part of the real system so there is no re-learning when the complete system is available.** 2. **Customers do not have to wait until the entire system is delivered before they can gain value from it.** The first increment satisfies their most critical requirements so they can use the software immediately. 3. *The process maintains the benefits of incremental development in that it should be relatively easy to incorporate changes...*"

ثلاث فوائد — والثانية الأولى مهمة جداً:

1. الزيادات المبكرة تُستخدم كنماذج أولية — العميل يكتسب خبرة تُغذي متطلباته للزيادات اللاحقة. والميزة الحاسمة: «بخلاف النماذج الأولية، هذي جزء من النظام الحقيقي — فلا يوجد إعادة تعلّم لما يتوفر النظام الكامل.»
هذي نقطة ذكية: النموذج الأولي يرمى → فالعميل يتعلّم شي يختفي. أما الزيادة فتبقى → لا إعادة تعلّم.
2. العميل ما ينتظر النظام كامل — الزيادة الأولى تحقق أهم متطلباته فيستخدم البرنامج فوراً.
3. العملية تحفظ فوائد التطوير التزايدي — فيسهل دمج التغييرات.

5.3 The three problems with incremental delivery

Verbatim (Sommerville p.65): "However, there are problems with incremental delivery: 1. **Most systems require a set of basic facilities that are used by different parts of the system.** As requirements are not defined in detail until an increment is to be implemented, it can be **hard to identify common facilities that are needed by all increments.** 2. *Iterative development can also be difficult when a replacement system is being developed. Users want all of the functionality of the old system and are often unwilling to experiment with an incomplete new system. Therefore, getting useful customer feedback is difficult.* 3. **The essence of iterative processes is that the specification is developed in conjunction with the software. However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract.** In the incremental approach, **there is no complete system specification until the final increment is specified.** This requires a **new form of contract**, which large customers such as **government agencies may find difficult to accommodate."**

ثلاث مشاكل:

#	المشكلة	التفصيل
1	صعوبة تحديد المرافق المشتركة	أغلب الأنظمة تحتاج مرافق أساسية تستخدمها أجزاء مختلفة. ولأن المطلوبات ما تُعرّف بالتفصيل إلا عند تنفيذ الزيادة، يصعب تحديد المرافق المشتركة المطلوبة بكل الزيادات
2	صعوبة عند استبدال نظام قائم	المستخدمون يريدون كل وظائف النظام القديم، وغير مستعدين يجزّون نظاماً جديداً ناقصاً → الحصول على تغذية راجعة مفيدة صعب
3	التعارض مع نموذج الشراء (procurement)	جوهر العمليات التكرارية أن المواصفة تُطوّر مع البرنامج — وهذا يتعارض مع نموذج الشراء بأغلب المؤسسات، حيث المواصفة الكاملة جزء من عقد تطوير النظام. وفي التزايدي ما توجد مواصفة كاملة لحد الزيادة الأخيرة → يحتاج شكل عقد جديد، وهذا يصعب على العملاء الكبار مثل الجهات الحكومية تقبله

المشكلة الثالثة هي الأعمق تنظيمياً: النموذج التقني يتعارض مع النموذج التعاقدية. وهذي مو مشكلة هندسية — مشكلة إدارية وقانونية. ولهذا التزايدي صعب بالقطاع الحكومي تحديداً.

5.4 And where incremental delivery simply does not fit

Verbatim (Sommerville p.65): "There are some types of system where incremental development and delivery is not the best approach. These are very large systems where development may involve teams working in different locations, some embedded systems where the software depends on hardware development and some critical systems where all the requirements must be analyzed to check for interactions that may compromise the safety or security of the system."

ثلاثة أنواع ما يناسبها التطوير والتسليم التزايدي:

النوع	السبب
أنظمة ضخمة جداً	فرق تعمل بمواقع مختلفة
أنظمة مضمّنة	البرنامج يعتمد على تطوير العتاد
أنظمة حرجة	كل المطلوبات لازم تُحلّل لفحص التفاعلات اللي قد تهدّد السلامة أو الأمن

وقد شفنا بالمفهوم السابق إن البديل لهذه الأنظمة هو... النموذج الأولي (Sommerville p.65). يعني النموذجان مكملان حسب نوع النظام.

6. The organisational obstacle

Sommerville adds a sidebar that is easy to skip and worth keeping, because it explains why a technically superior model can fail in practice.

Verbatim (Sommerville p.51): "Although incremental development has many advantages, it is not problem-free. The primary cause of the difficulty is the fact that large organizations have bureaucratic procedures that have evolved over time and there may be a mismatch between these procedures and a more informal iterative or agile process."

Verbatim (Sommerville p.51): "Sometimes these procedures are there for good reasons — for example, there may be procedures to ensure that the software properly implements external regulations (e.g., in the United States, the Sarbanes-Oxley accounting regulations). Changing these procedures may not be possible so process conflicts may be unavoidable."

السبب الأساسي للصعوبة: المؤسسات الكبيرة عندها إجراءات بيروقراطية تطوّرت مع الزمن، ويمكن يصير عدم توافق بين هذي الإجراءات وعملية تكرارية أو رشيقة أكثر مرونة.

وبعض الإجراءات موجودة لأسباب وجيهة — مثلاً ضمان تطبيق التشريعات الخارجية (مثل قوانين Sarbanes-Oxley للمحاسبة بأمريكا). وتغيير هذي الإجراءات قد يكون مستحيلاً — فالتعارضات العملية لا مفرّ منها.

ليش هذي مهمة؟ لأنها تعطي إجابة على سؤال «ليش ما كل الشركات تستخدم التزايدي مع إنه أفضل؟» — الجواب: لأن العملية التقنية محبوسة بالإجراءات التنظيمية والتشريعية، ومو دالماً ممكن تغييرها.

وهذا ينفع جداً بالسيناريوهات: لو السؤال وصف بنكاً أو جهة حكومية أو مؤسسة منظمّة، فالجواب لازم يذكر التعارض مع إجراءات الشراء والتنظيم.

7. Where this model sits in the series

Two axes now apply, and incremental scores on both.

Axis	Where incremental sits
Change avoidance vs tolerance (Unit 03 §1)	Both. Sommerville: " <i>This supports both change avoidance and change tolerance</i> " (p.61) — avoidance because early increments teach the requirements, tolerance because changes land in increments not yet built
Waterfall shortcomings repaired (Unit 02 §3.7)	Long delivery → repaired (benefit 3) · limited customer interactions → repaired (benefit 2) · no phase overlap → repaired (activities are interleaved)
Waterfall shortcomings NOT repaired	Heavy documentation → not addressed · no support for risk handling → not addressed · error correction cost → partly addressed
What it costs you	The process is not visible · the structure degrades · it is slow (you wait for the last version)

على المحورين:

1. تجنّب/تحمل التغيير: الاثنان معاً — وهذا وحيد بين النماذج اللي شفناها.

2. العيوب اللي يصلحها من الـ Waterfall:

يصلّحه؟	العيوب
نعم — الفائدة 3	التسليم الطويل
نعم — الفائدة 2	تفاعل محدود مع العميل
نعم — الأنشطة متداخلة (interleaved)	ما يدعم تداخل المراحل

3. العيوب اللي ما يصلحها:

الحكم	العيوب
لا — بل يعكس المشكلة: صار ناقصاً	التوثيق المفرط
لا — وهذا شغل Spiral	ما يدعم المخاطر

4. والكلفة اللي يفرضها: العملية غير مرئية · البنية تتدهور · وبطيء.

والنقطة الأخيرة هي بوابة الملف 05: لأن العميل ينتظر الزيادة الأخيرة لياخذ النظام الكامل. فلو الجدول الزمني هو القيد الحقيقي — لازم نموذج يضغط الوقت. وهذا بالضبط RAD.

Source notes

Source	Verdict for this file
Sommerville pp.49–51	Primary for §1, §2.1, §3.1, §4 and §5 (the development/delivery distinction). The definition, the interleaving, the "reflects how we solve problems" argument, the three benefits, the two management problems, the large-system caveat, the development-versus-delivery distinction, and the organisational obstacle sidebar. Mall has none of the management problems, none of the benefits, and none of the development/delivery distinction.

Source	Verdict for this file
Sommerville pp.64–65	Primary for §5.1–5.4. Incremental delivery in full — the priority-based allocation, the local freezing rule, the three advantages (including "no re-learning"), the three problems (including the procurement conflict), and the three system types where it does not fit.
Mall pp.95–97	Primary for §2.2 and §3.2. The second name (<i>successive versions model</i>), the "simple working system delivered" definition, the core / non-core distinction , the rule that each version is built with iterative waterfall , the feedback loop, and the two advantages — error reduction and incremental resource deployment . Sommerville has neither advantage.
Mall pp.94–95	The prototyping strengths/weaknesses balance sheet. Recorded in Unit 03 §3.4 , where it belongs — it is what closes the prototyping topic and opens this one.
Pressman / Agarwal	Add little. Both cover incremental development at a lower level of precision and neither adds a distinction absent from Sommerville and Mall.
Coverage note	Mall's incremental model section is short (pp.95–97). Sommerville is the deeper source here — the reverse of Files 02 and 03, where Mall was primary. Worth noting because it means this file's centre of gravity is Sommerville.
Note	The three benefits differ between the sources and both sets are examinable. Sommerville: reduced cost of change · easier customer feedback · more rapid delivery. Mall: error reduction · incremental resource deployment. They do not overlap. Learn both, and if the question says "according to...", answer that source's set.

Retrieval set

Answers appear directly beneath each question, as agreed. **Cover the answer, produce your own, then compare.**

1 Define incremental development, and name the second term Mall uses for it.

"Developing an initial implementation, exposing this to user comment and evolving it through several versions until an adequate system has been developed." Mall calls it also the **successive versions model**, and defines it as: "first a simple working system implementing only a few basic features is built and delivered to the customer. Over many successive iterations successive versions are implemented and delivered... until the desired system is realised." (Sommerville p.49; Mall p.95)

2 What does Sommerville mean when he says the activities are "interleaved"?

That "specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities." The waterfall keeps them as distinct stages; incremental runs them concurrently. (Sommerville p.50)

3 Give the argument that incremental development is natural.

"Incremental development reflects the way that we solve problems. We rarely work out a complete problem solution in advance but move toward a solution in a series of steps, backtracking when we realize that we have made a mistake." (Sommerville p.50)

4 State Sommerville's three benefits, and say which is the deepest and why.

(1) The cost of accommodating changing requirements is reduced — less analysis and documentation has to be redone. **(2)** It is easier to get customer feedback — and *"customers find it difficult to judge progress from software design documents."* **(3)** More rapid delivery and deployment of usable software is possible. **Benefit 2 is the deepest**, because it says documents are not an effective communication channel with the customer — which is why later models replace documents with working software. (Sommerville p.50)

5 State Mall's two advantages, and say how they differ from Sommerville's.

(1) Error reduction — the core modules *"are used by the customer from the beginning and therefore these get tested thoroughly"*, reducing errors in the core of the final product and giving *"greater reliability."* **(2) Incremental resource deployment** — it *"obviates the need for the customer to commit large resources at one go"*, and saves the developer from deploying large resources and manpower at once. **They do not overlap with Sommerville's three at all** — Mall's are about **early testing** and **financing**, not about cost of change, feedback or delivery speed. (Mall p.97)

6 How does Mall decide which features go into the first version?

The **core features** go first, and the definition is precise: *"core or basic features are those that do not need to invoke any services from the other features. On the other hand, non-core features need services from the core features."* So core = independent. (Mall p.96)

7 What model is used inside each increment, and what does that tell you about how these models relate?

"Each incremental version is usually developed using an iterative waterfall model of development." It tells you the models are **not mutually exclusive** — the incremental model **contains** the iterative waterfall. (Mall p.96)

8 State the two management problems with incremental development.

(1) The process is not visible — managers need regular deliverables to measure progress, but *"if systems are developed quickly, it is not cost-effective to produce documents that reflect every version."* **(2) System structure tends to degrade** — *"unless time and money is spent on refactoring... regular change tends to corrupt its structure",* and further changes become *"increasingly difficult and costly."* (Sommerville p.51)

9 For which kind of system do these problems become acute, and what must therefore be planned in advance?

Large, complex, long-lifetime systems with different teams on different parts. They need a **stable framework or architecture**, and team responsibilities must be clearly defined against it — and *"this has to be planned in advance rather than developed incrementally."* So: plan the architecture, develop the content incrementally. (Sommerville p.51)

10 Distinguish incremental development from incremental delivery.

With **incremental development** you can *"expose it to customers for comment, without actually delivering it and deploying it in the customer's environment."* With **incremental delivery and deployment** the software *"is used in real, operational processes."* The second is not always possible because *"experimenting with new software can disrupt normal business processes."* (Sommerville p.51)

11 In incremental delivery, how are services allocated to increments, and what is frozen?

Customers identify the services *"in outline"* and rank them; allocation *"depends on the service priority, with the highest-priority services implemented and delivered first."* What is frozen is local: *"further requirements analysis for later increments can take place but *requirements changes for the current increment are not accepted."* (Sommerville p.64)

12 Give the three advantages of incremental delivery, and explain the "no re-learning" point.

(1) Early increments act as prototypes and inform later requirements — and *"unlike prototypes, these are part of the real system so there is no re-learning when the complete system is available."* **(2)** Customers *"do not have to wait until the entire system is delivered before they can gain value from it."* **(3)** The benefits of incremental development are maintained. **The "no re-learning" point** is that a throwaway prototype teaches the customer something that then disappears; a delivered increment teaches the same thing and **stays**. (Sommerville p.64)

13 Give the three problems with incremental delivery, and say which is organisational rather than technical.

(1) Common facilities needed by all increments are **hard to identify**, since requirements are only detailed when an increment is implemented. (2) It is **difficult when replacing an existing system** — users want "all of the functionality of the old system" and are "unwilling to experiment with an incomplete new system." (3) It **conflicts with the procurement model** — "the complete system specification is part of the system development contract", but in the incremental approach "there is no complete system specification until the final increment is specified." **Problem 3 is organisational**, not technical — it "requires a new form of contract, which large customers such as government agencies may find difficult to accommodate." (Sommerville p.65)

14 Name the three kinds of system where incremental development and delivery is not the best approach.

Very large systems (teams in different locations), some **embedded systems** (software depends on hardware development), and some **critical systems** (all requirements must be analysed for interactions compromising safety or security). (Sommerville p.65)

15 What is the primary cause of difficulty in adopting incremental development, according to Sommerville?

That "large organizations have bureaucratic procedures that have evolved over time" and there may be a "mismatch between these procedures and a more informal iterative or agile process." Some procedures exist for good reasons — e.g. compliance with **external regulations such as Sarbanes-Oxley** — and "changing these procedures may not be possible so process conflicts may be unavoidable." (Sommerville p.51)

16 Is incremental development plan-driven or agile, and how common is it?

Both — "this approach can be either plan-driven, agile, or, more usually, a mixture of these approaches." In a **plan-driven** approach the increments are "identified in advance"; in an **agile** approach "the early increments are identified but the development of later increments depends on progress and customer priorities." And: "incremental development in some form is now the most common approach for the development of application systems." (Sommerville p.51)

Page anchors

Revisit	For
Sommerville p.47	The three generic models and the rule that well-understood parts use waterfall while unclear parts use an incremental approach
Sommerville p.49	§2.1.2 opens — the definition of incremental development

Revisit	For
Sommerville p.50	The interleaving of activities · <i>"reflects the way that we solve problems"</i> · the early increments carry the most important functionality · the three benefits · Figure 2.2
Sommerville p.51	Most common approach · plan-driven/agile/mixture · the two management problems · the large-system caveat · the development-versus-delivery distinction · the bureaucratic-mismatch sidebar (Sarbanes-Oxley)
Sommerville p.64	§2.3.2 Incremental delivery — priority-based allocation · <i>"requirements changes for the current increment are not accepted"</i> · the three advantages , including "no re-learning"
Sommerville p.65	The three problems with incremental delivery, including the procurement conflict · the three system types where it does not fit
Mall p.94–95	Prototyping strengths and weaknesses (now in Unit 03 §3.4) — and the point that prototyping is <i>"ineffective for risks identified later during the development cycle"</i> , which opens this file
Mall p.95	§2.2.5 opens — the <i>successive versions</i> name · the "simple working system delivered" definition · Figure 2.7
Mall p.96	Requirements split into versions · core vs non-core features · <i>"each incremental version is usually developed using an iterative waterfall model"</i> · the feedback loop and the two directions of refinement · Figure 2.8
Mall p.97	The two advantages — error reduction and incremental resource deployment · §2.2.6 opens